

CS 1671 / CS 2071

# Human Language Technologies

Session 12: Vector semantics, word2vec

---

Michael Miller Yoder

October 1, 2026



University of  
Pittsburgh

School of Computing and Information

# Assessments

- [Project deliverable 1](#) is due **next Thu Oct 8**
- [Homework 2](#) on text classification was released today and is due **Oct 15**

# Structure of this course

## MODULE 1

### Prerequisite skills for NLP

text normalization, linear alg., prob., machine learning

## MODULE 2

### Approaches

statistical machine learning

### How text is represented

n-grams

### NLP tasks

language modeling  
text classification

## MODULE 3

neural networks

static word vectors

text classification

## MODULE 4

language modeling  
text classification  
sequence labeling

## MODULE 5

### NLP applications and ethics

# Overview: vector semantics, static word embeddings

- Coding activity: clickbait classification evaluation and error analysis
- Vector semantics
- Distributional semantics
- Types of word vectors
- Word2vec
- Bias in word vectors
- Coding activity: explore word vectors

# Harms in classification in NLP

---

# Harms in sentiment classifiers

Kiritchenko and Mohammad (2018) found that most sentiment classifiers assign lower sentiment and more negative emotion to sentences with African American names in them.

This perpetuates negative stereotypes that associate African Americans with negative emotions

# Harms in toxicity classification

Toxicity detection is the task of detecting hate speech, abuse, harassment, or other kinds of toxic language

But some toxicity classifiers incorrectly flag as being toxic sentences that are non-toxic but simply mention identities like blind people, women, or gay people.

This could lead to censorship of discussion about these groups.

# What causes these harms?

Can be caused by:

- Problems in the training data; machine learning systems are known to amplify the biases in their training data.
- Problems in the human labels
- Problems in the resources used (like lexicons)
- Problems in model architecture (like what the model is trained to optimized)

Mitigation of these harms is an open research area

Can't fully “remove” bias because exists in societies that produced texts we use

So need to be explicit about what those biases may be through **data statements** and **model cards**

# Data statements [Bender & Friedman 2018]

For each dataset you release, document:

- Curation rationale: why were certain texts selected
- Language variety
- Speaker demographic
- Annotator demographic
- Speech situation
  - Time and place, modality, scripted vs spontaneous, intended audience
- Text characteristics
  - Genre, topic
- Recording quality (for speech)

# Model cards [Mitchell et al. 2019]

For each algorithm you release, document:

- training algorithms and parameters
- training data sources, motivation, and preprocessing
- evaluation data sources, motivation, and preprocessing
- intended use and users
- model performance across different demographic or other groups and environmental situations

# Discussion: data statements and model cards

- In an area of research or industry you are familiar with (in CS or outside CS), do you see issues with transparency of data and/or models?
  - Is it clear under what circumstances datasets were collected?
  - Are the intended uses of machine learning models clearly stated?
- What are limitations of data statements and model cards?

# Vector semantics

---

# Semantics: the study of meaning

Word representations in NLP draw on 2 areas of semantics

- a. Vector semantics
- b. Distributional semantics

# Vector semantics

Modeling semantics as points in vector space

- Words or other text segments are represented by vectors
- Multiple dimensions
- Nearer = more similar words

# Term-document matrix: word vectors

|                                                                                   |                       | battle | soldier | fool | wit |
|-----------------------------------------------------------------------------------|-----------------------|--------|---------|------|-----|
|  | <i>As You Like It</i> | 1      | 2       | 36   | 20  |
|  | <i>Twelfth Night</i>  | 0      | 2       | 58   | 15  |
|  | <i>Julius Caesar</i>  | 7      | 12      | 1    | 2   |
|  | <i>Henry V</i>        | 13     | 36      | 4    | 3   |

- Each cell is the count of term  $t$  in a document  $d$  ( $\text{tf}_{t,d}$ )
- Each document is a count vector in  $\mathbb{N}^V$ , a row above

Pairs of similar words?

# Similarity and relatedness

- Synonyms: big/large, couch/sofa, automobile/car
- Similar: sharing some element of meaning
  - coffee/tea, car/bicycle, cow/horse
- Related: by a semantic field
  - coffee/cup, scalpel/surgeon



# Distributional semantics

---

# Distributional semantics

"The meaning of a word is its use in the language" [Wittgenstein 1953]



"You shall know a word by the company it keeps" [Firth 1957]



"If A and B have almost identical environments we say that they are synonyms" [Harris 1954]



# Distributional semantics

Define the meaning of a word by its **distribution** in language use: its neighboring words or grammatical environments.

# You Learn Words by Using Distributional Similarity



Consider

- A bottle of pocarisweat is on the table.
- Everybody likes pocarisweat.
- Pocarisweat makes you feel refreshed.
- They make pocarisweat out of ginger.

What does *pocarisweat* mean?

# You Know Pocarisweat by the Company It Keeps



From context words humans can guess *pocarisweat* means a beverage like **coke**.

How do you know?

- Other words can occur in the same context
- Those other words are often for beverages (that you drink cold)
- You assume that *pocarisweat* is probably similar

So the intuition is that **two words are similar if they have similar word contexts**.

# You Can Tell a Lot about *Beef* from Its Contexts

1 fertility. Organ meats such as beef and chicken liver, tongue and hear  
2 controlling scours. HOW TO FEED: BEEF AND DAIRY CALVES - 0.2 gram Dy  
3 ing process discolors the treated beef and liquid accumulates in prepackag  
4 say. He did say she could get her beef and vegetables in cans this summer  
5 and feed efficiency of fattening beef animals. HOW TO FEED: At the  
6 steaks, chops, chicken and prime beef as well as Tom's favorite dish, stu  
7 ross from him was surmounted by a beef barrel with ends knocked out. In t  
8 counter of boards laid across two beef barrels. There was, of course, no  
9 Because Holstein cattle weren't a beef breed, they were rarely seen on a  
10 2-5 grams of phenothiazine daily; beef calves- .5 to 1.5 grams daily depe  
11 ties of this drug. HOW TO FEED: BEEF CATTLE (FINISHING RATION) - To  
12 dairy cows and lesser amounts to beef cattle and poultry. About 90 percen  
13 raises enough poultry, pigs, and beef cattle for most of their needs. Lo  
14 on of liver abscesses in feed-lot beef cattle. Prevention of bacterial pne  
15 pal feed bunk types for dairy and beef cattle: (1) Fence-line bunks- catt  
16 es feed efficiency. HOW TO FEED: BEEF CATTLE - 10 milligrams of diet  
17 the rations you are feeding your beef, dairy cattle, and sheep are adequa  
18 itive business more profitable for beef, dairy, and sheep men. The tar  
19 o bear. She was ready to kill the beef, dress it out, and with vegetables  
20 . She had raised a calf, grown it beef-fat. She had, with her own work-wea  
21 with feeding low-moisture corn in beef-feeding programs. Several firms ar  
22 he shelf life (at 35 F) of fresh beef from 5 days to 5 or 6 weeks. Howeve  
23 canned pork products. Tests with beef have been largely unsuccessful beca  
24 for eggs, pigs to eat garbage, a beef herd and wastes of all kinds. Separ  
25 their money's worth. A good many beef-hungry settlers were accepting the

## Contexts for *Chicken* Are also Informative

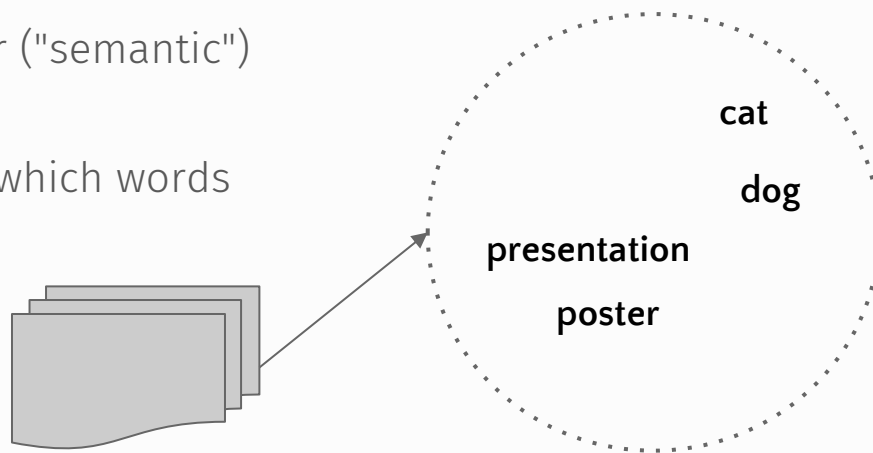
1 y the irradiated and refrigerated chicken. Acceptance of radiopasteurization  
2 torehouse". Glendora dropped a chicken and a flurry of feathers, and went  
3 will specialize in steaks, chops, chicken and prime beef as well as Tom's fa  
4 ard as the one concerned with the chicken and the egg. Which came first? Is  
5 he millions of buffalo and prairie chicken and the endless seas of grass that  
6 "! "Come on, there's some cold chicken and we'll see what else". They wen  
7 ves to extend the storage life of chicken at a low cost of about 0.5 cent per  
8 CHICKEN CADILLAC# Use one 6-ounce chicken breast for each guest. Salt and pe  
9 ion juice, to about half cover the chicken breasts. Bake slowly at least one-  
10 d, in butter. Sprinkle over top of chicken breasts. Serve each breast on a th  
11 around, they had a hard time". #CHICKEN CADILLAC# Use one 6-ounce chicken  
12 successful, and the shelf life of chicken can be extended to a month or more  
13 ay from making a cake, building a chicken coop, or producing a book, to found  
14 , they decided, but a deck full of chicken coops and pigpens was hardly suita  
15 im. "Johnny insisted on cooking a chicken dinner in my honor- he's always bee  
16 nutes. Kid Ory, the trombonist chicken farmer, is also one of the solid a  
17 y Johnson reaching around the wire chicken fencing, which half covered the tr  
18 yes glittering behind dull silver chicken fencing. "That was Tee-wah I was t  
19 wine in the pot roast or that the chicken had been marinated in brandy, and  
20 yed this same game and called it "Chicken". He could not go through the f  
21 f the Mexicans hiding in a little chicken house had passed through his head,  
22 I'll never forget him cleaning the chicken in the tub". A story, no doubt  
23 . Organ meats such as beef and chicken liver, tongue and heart are planne  
24 p. "Miss Sarah, I can't cut up no chicken. Miss Maude say she won't". Aga

# Types of word vectors

---

# Shared Intuition: Words are Vectors of Numbers Representing Meaning

- Model the meaning of a word by “**embedding**” it in a vector space.
- The meaning of a word is a vector of numbers:
  - Vector models are also called **embeddings**
  - Often, the word *embedding* is reserved for *dense* vector representations
- In contrast, word meaning is represented in many (early) NLP applications by a vocabulary index (“word number 545”; compare to **one-hot representations**)
- Similar words are nearby in vector ("semantic") space
- Build "semantic space" by seeing which words are nearby in text



# There are Two Kinds of Vector Models

- **Sparse embeddings** (vectors from term-document matrix)
  - long (length of 20,000 to 50,000)
  - sparse: most elements are 0
- **Dense embeddings** (Word2vec)
  - short (length of 50-1000)
  - dense (most elements are non-zero)

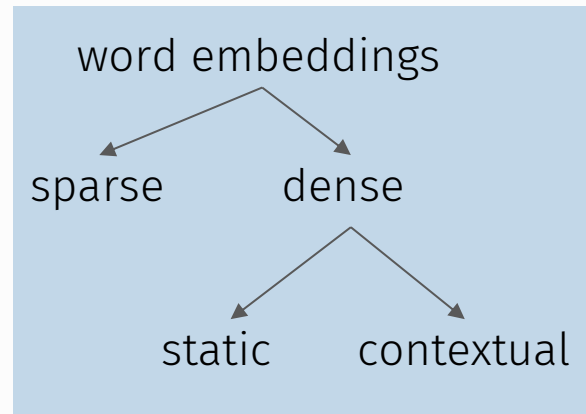


# Dense Vectors Have Three Advantages over Sparse Vectors

1. Short vectors may be **easier to use as features** in machine learning (less weights to tune).
2. Dense vectors may **generalize better** than storing explicit counts.
3. They may do **better at capturing synonymy**:
  - *car* and *automobile* are synonyms
  - But, in sparse vectors, they are represented as distinct dimensions
  - This fails to capture similarity between a word with *car* as a **neighbor** and a word with *automobile* as a **neighbor**

# Methods for learning short, dense word embeddings

- Static, neural embeddings
  - Fixed embeddings for word types
  - Word2Vec, GloVe
- Contextual embeddings
  - Embeddings for words vary by context
  - ELMo, BERT, LLMs



# Word2vec

---

# Word2vec [Mikolov et al. 2013]

- Instead of counting words, train a classifier on a binary prediction task
  - Is  $w_1$  likely to show up near  $w_2$ ?

# Word2vec [Mikolov et al. 2013]

- Instead of counting words, train a classifier on a binary prediction task
  - Is  $w_1$  likely to show up near *apricot*?



# Word2vec [Mikolov et al. 2013]

- Instead of counting words, train a classifier on a binary prediction task
  - Is  $w_1$  likely to show up near *apricot*?
- Take the learned classifier weights as the word embeddings



# Word2vec [Mikolov et al. 2013]

- Instead of counting words, train a classifier on a binary prediction task
  - Is  $w_1$  likely to show up near *apricot*?
- Take the learned classifier weights as the word embeddings
- Training techniques: skip-gram and CBOW



# Word2vec: training supervision

- **Self-supervision** [Bengio et al. 2003, Collobert et al. 2011]
- Use naturally occurring text as labels
- A word  $c$  that occurs near *apricot* in the corpus counts as the gold "correct answer" for supervised learning

# Word2vec training overview

1. Positive examples: the target word  $w$  and a neighboring context word  $c_{pos}$
2. Negative examples: Randomly sample other words  $c_{neg}$  in the lexicon to pair with  $w$
3. Use logistic regression to train a classifier to distinguish those two cases
4. Use the learned weights ( $W, C$ ) as the word embeddings

# Training for Embeddings

- We do not know what  $W$  and  $C$  are. So we learn them through an iterative process.
- We use a large corpus as a training data
- We also randomly sample the corpus to find words that are NOT in the context—negative sampling.



| Positive Examples |        | Negative Examples |          |      |         |
|-------------------|--------|-------------------|----------|------|---------|
| t                 | c      | t                 | c        | t    | c       |
| ides              | beware | ides              | aardvark | ides | twelve  |
| ides              | of     | ides              | puddle   | ides | hello   |
| ides              | March  | ides              | where    | ides | dear    |
| ides              | the    | ides              | coaxial  | ides | forever |

# Word2vec: learning embeddings

- Start with randomly initialized context  $C$  and target word  $W$  matrices
- Go through the positive and negative training pairs, adjusting word vectors such that we:
  - Maximize the similarity of the target word, context word pairs  $(w, c_{pos})$  drawn from the positive data
  - Minimize the similarity of the  $(w, c_{neg})$  pairs drawn from the negative data.

# Skip-gram classifier

Classifier input pairs:

(target word  $w$ , context word  $c$ )

Classifier output: probabilities that  $w$  occurs with  $c$

$$P(+|w, c)$$

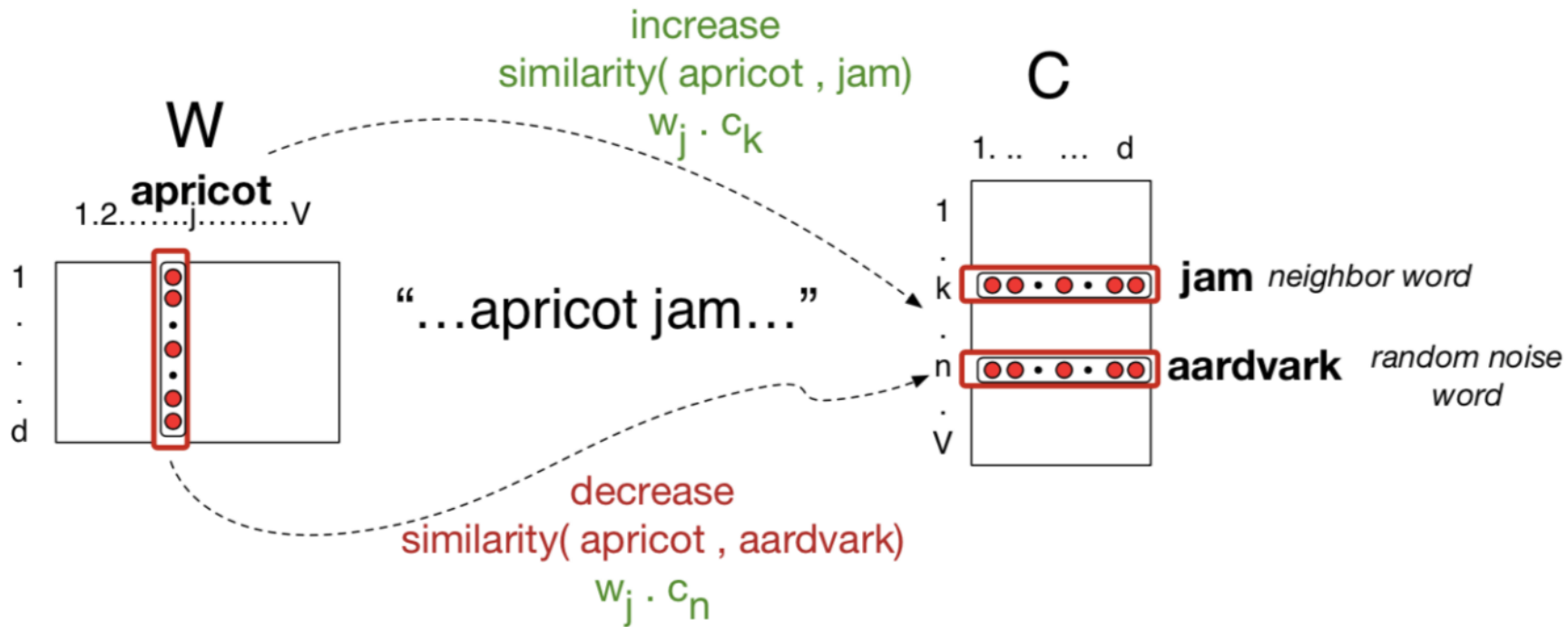
$$P(-|w, c) = 1 - P(+|w, c)$$

# Skip-gram classifier: calculating probabilities

- From input vectors, need to compare for similarity
- Start with dot product:  $\text{sim}(\mathbf{w}, \mathbf{c}) \approx \mathbf{w} \cdot \mathbf{c}$
- To turn this into a probability, use the sigmoid function from logistic regression:

$$P(+|w, c) = \sigma(c \cdot w) = \frac{1}{1 + \exp(-c \cdot w)}$$

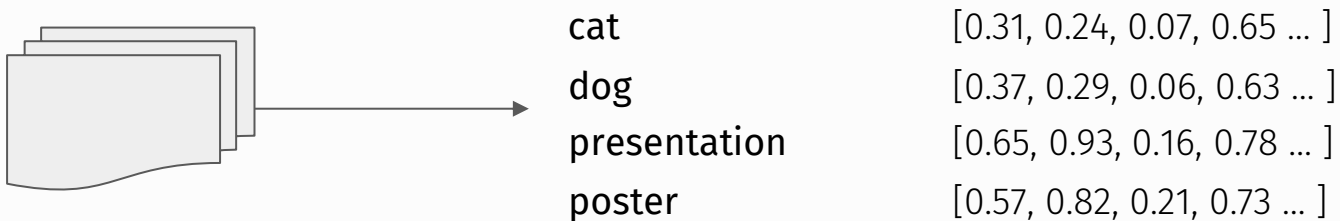
# Training for Embeddings



# Reminder: one step of gradient descent

- Direction: We move in the reverse direction from the gradient of the loss function
- Magnitude: we move the value of this gradient  $d/dw L(P(+|w,c) + P(-|w,c))$  weighted by a learning rate  $\eta$
- Higher learning rate means move  $w$  faster

# Summary: How to learn word2vec embeddings

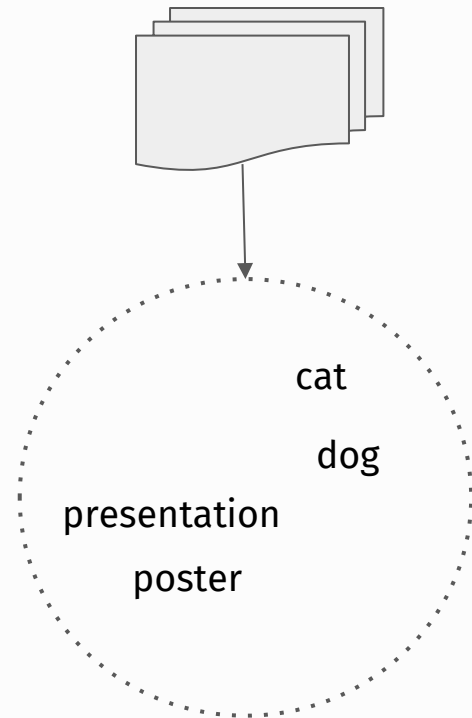


# Summary: How to learn word2vec embeddings

1. Start with randomly initialized word embeddings
2. From a corpus, extract pairs of words that co-occur (positive)
3. Extract pairs of words that don't co-occur (negative)
4. Train a classifier to distinguish between positive and negative examples by slowly adjusting all the embeddings to improve the classifier performance
5. Keep the weights as our word embeddings

# Final embeddings

- Can add representations for a word in  $W$  and in  $C$  together for final word vector for  $W_i$
- Can just keep  $W$  and throw away  $C$
- Can find "nearest neighbors" of certain words with cosine similarity in embedding space



# There are Tools and Resources Available for Training and Using Embeddings

- **Pretrained embeddings**
  - Skip-gram
  - CBOW
  - fastText
  - GloVe
- **Training your own embeddings**
  - You can easily train skip-gram, CBOW, and fastText embeddings with `gensim`
  - Straightforward Python interface

# Embeddings reflect cultural biases [Bolukbasi et al. 2016]

- Paris : France :: Tokyo : *Japan*
- Sexist occupational stereotypes
  - father : doctor :: mother : *nurse*
  - man : computer programmer :: woman : *homemaker*
- Would be problematic to use embeddings in hiring searches for programmers

# Conclusion: vector semantics, static word embeddings

- NLP typically represents words as vectors in spaces where distance  $\approx$  semantic similarity
- Word2vec learns static embeddings (vectors) for words by predicting which words occur together in training data
- These embeddings are effective in downstream NLP tasks, but also reflect social biases of training data

Coding activity:  
examine word2vec embeddings

---

# Notebook: examine word2vec embeddings

1. Go to this [nbgitpuller link](#) (also available on course website)
2. Start a server with **TEACH – 6 CPUs – 45 GB**
3. Load custom environment at  
`/ix1/cs1671-2026f/class-env/.pixi/envs/default`
4. Open **session12\_word2vec.ipynb**